UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Universidad Distrital Francisco José de Caldas
Systems engineering, school of Engineering

Technical Report: Game Variant Fire Brigade

R. Andres Diaz Lozada, A. Mauricio Cepeda Villanueva, J. Felipe Cuervo Huertas

Supervisor: Carlos A. Sierra Virgüez

March 27, 2026

Declaration

We, R. Andres Diaz Lozada, A. Mauricio Cepeda Villanueva, J. Felipe Cuervo Huertas of Systems Engineering, Faculty of Engineering, Francisco José de Caldas District University, confirm that this is our own work, and that figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where such works have been explicitly acknowledged, quoted, and referenced. We understand that failing to do so will be considered a case of plagiarism and penalized accordingly. We give consent to a copy of our report being shared with future students as an exemplar. We give consent for our work to be made available more widely to members of Universidad Distrital Francisco José de Caldas and to the public with interest in teaching, learning, and research.

R. Andres Diaz Lozada, A. Mauricio Cepeda Villanueva, J. Felipe Cuervo Huertas
March 27, 2026

Abstract

This report presents the design and development of Fire Brigade, a turn-based simulation game that models forest fire suppression and civilian evacuation on an 8×8 grid. The project implements two fundamental algorithms: a Greedy algorithm for automatic firefighting unit dispatch to the highest-risk cell, and a Backtracking algorithm for finding safe evacuation routes avoiding burning cells. The technical architecture separates the game engine (C++) from the algorithmic logic (Python), communicating via JSON and presenting the interface through Pygame. Key design decisions include the use of a Binary Search Tree (BST) to maintain cell risk levels for O(log n) access, and a Linked List to manage the unit dispatch queue. The game incorporates three difficulty levels, three unit types (Crew, Truck, Helicopter), four civilian types (Child, Adult, Elderly, Family Group), and two fire behaviors (Normal and Canopy). This report details the system architecture, data structures, algorithms, and the expected outcomes for the complete implementation.

Keywords: Greedy Algorithm, Backtraking, Binary Search Tree (BST), Fire Suppression, Civilian Evacuation, Python, PyGame, C++, Data Structure, Linked Lists.
GitHub repository `enlace`

Contents

List of Tables v

1 Introduction 1

2 Literature Review 2
- 2.1 Greedy Algorithms . . . 2
- 2.2 Backtracking Search . . . 2
- 2.3 Linked Lists . . . 3
- 2.4 Binary Search Trees . . . 3
- 2.5 Summary . . . 3

3 Objectives 4
- 3.1 General Objective . . . 4
- 3.2 Specific Objectives . . . 4

4 Scope 5
- 4.1 Included in this Study . . . 5
- 4.2 Excluded from this Study . . . 5

5 Assumptions and Limitations 7
- 5.1 Assumptions . . . 7
- 5.2 Limitations . . . 7

6 Methodology 9
- 6.1 System Architecture . . . 9
- 6.2 Data Structure Design . . . 9
  - 6.2.1 Unit Dispatch Queue — Singly Linked List . . . 9
  - 6.2.2 Risk-Ranking Structure — Binary Search Tree . . . 10
- 6.3 Algorithm Design . . . 10
  - 6.3.1 Greedy Unit Dispatcher . . . 10
  - 6.3.2 Backtracking Evacuation Planner . . . 10
- 6.4 Game Rules and Difficulty Levels . . . 11
  - 6.4.1 Scoring System . . . 11
  - 6.4.2 Win and Loss Conditions . . . 11
  - 6.4.3 Difficulty Levels . . . 12
- 6.5 Turn Structure . . . 12

7 Implementation & Results 13

8 Conclusions 14

Appendices 15

References **15**

List of Tables

6.1 Difficulty level configurations 12

List of Abbreviations

BST	Binary Search Tree
FIFO	First-In-First-Out
GUI	Graphical User Interface
HUD	Heads-Up Display
JSON	JavaScript Object Notation
O(n)	Big O Notation (linear time complexity)
O(log n)	Big O Notation (logarithmic time complexity)
UI	User Interface

Chapter 1

Introduction

This report presents the design and implementation of a turn-based tactical simulation game set on an 8×8 grid, in which the player commands firefighting and civilian evacuation operations. The game was developed as an academic project to demonstrate the practical application of fundamental data structures and algorithms within an interactive software system.

The project integrates three distinct technological layers: a C++ engine responsible for core data management, a Python module housing the decision-making algorithms, and a Pygame-based graphical interface. Communication between the C++ engine and the Python layer is handled through a JSON Bridge, allowing each component to operate independently while sharing game state efficiently.

At the core of the gameplay mechanics lie two mandatory algorithms. The **Greedy algorithm** drives unit deployment by consulting a Binary Search Tree (BST) maintained by the C++ engine, which ranks all burning cells by risk level and automatically dispatches the selected unit to the highest-risk cell. The **Backtracking algorithm** handles civilian evacuation by exhaustively exploring all possible safe paths from a civilian's position to the grid boundary, avoiding cells currently on fire, and returning the shortest valid route found.

The C++ engine manages two key data structures: a **singly linked list** that implements the unit dispatch queue, storing incoming firefighting units (ground crews, trucks, and helicopters) in order of arrival; and a **Binary Search Tree** that organises burning cells by their computed risk score, enabling the Greedy algorithm to retrieve the maximum-risk target in $O(\log n)$ time.

The game features three difficulty levels — Easy, Medium, and Hard — each varying the number and type of fires, the composition of civilians on the map, the initial unit allocation, and the rate at which new units arrive. Three built-in variants add strategic depth: a *scarce resources* variant governing unit types and arrival timing, a *civilian priorities* variant assigning different point values and movement speeds to four civilian categories, and a *dynamic fire behaviour* variant introducing two fire propagation modes with distinct spread patterns and risk weightings.

The development team was composed of three members, each responsible for a distinct layer of the architecture:

- **Jeison Felipe Cuervo Huertas** — Engine Developer: implemented the C++ core, including the linked list dispatch queue and the BST risk-ranking structure, as well as the JSON Bridge connecting the engine to the Python layer.
- **Andres Mauricio Cepeda Villanueva** — Algorithm Developer: implemented the Greedy dispatcher and the Backtracking evacuator in Python, including tie-breaking logic and path optimisation.
- **Raul Andres Diaz Lozada** — UI Developer: built the Pygame interface, rendering the 8×8 grid, fire propagation animations, evacuation sequences, and the score and resource HUD.

Chapter 2

Literature Review

This chapter reviews the theoretical foundations underlying the two core algorithms implemented in the project — the Greedy strategy and Backtracking search — as well as the data structures that support them: the singly linked list and the Binary Search Tree. Understanding the established literature on these topics justifies the design decisions made during development and contextualises the project within the broader field of algorithm design and data structure theory.

2.1 Greedy Algorithms

A greedy algorithm constructs a solution by making the locally optimal choice at each step, with the expectation that a sequence of local optima will converge toward a global optimum [1]. The defining characteristic of greedy strategies is their irrevocability: once a decision is made, it is never reconsidered. This property makes greedy algorithms computationally efficient, typically achieving $O(n \log n)$ or better time complexity, but also means they are not universally optimal — their correctness depends on whether the problem exhibits the *greedy-choice property* and *optimal substructure* [2].

In the context of this project, the **DespachadorGreedy** algorithm applies the greedy principle to unit deployment: at every dispatch decision, the unit is unconditionally sent to the cell with the highest current risk score as reported by the BST. This approach prioritises the most dangerous active fire at all times, reflecting a locally optimal strategy for damage containment. While this does not guarantee a globally optimal sequence of deployments across the entire game — since fire propagation introduces dynamic changes to the risk landscape — it provides a consistent, fast, and explainable decision rule that is well-suited to a real-time tactical setting.

Tie-breaking is handled by a secondary greedy rule: among cells sharing the maximum risk value, crown fire cells (fuego de copas) are preferred due to their higher propagation rate, and among remaining ties, the cell with the smallest row index followed by the smallest column index is selected, ensuring deterministic behaviour.

2.2 Backtracking Search

Backtracking is a general algorithm design paradigm for solving constraint satisfaction and combinatorial search problems [3]. It performs a depth-first exploration of the solution space, incrementally building candidates and abandoning (backtracking from) any partial candidate that cannot lead to a valid or optimal solution. This avoids the combinatorial explosion of exhaustive enumeration by pruning infeasible branches early [4].

The **EvacuacionBacktracking** algorithm implemented in this project applies backtracking to find the shortest safe evacuation route for a civilian on the 8×8 grid. Starting from the civilian's current cell, the algorithm performs a depth-first search across the four cardinal directions, marking visited cells to prevent cycles, and recording the path length each time the grid boundary is reached. Upon finding a complete path, it compares the path length against the current best and retains

the shorter one. After fully exploring a branch, it unmarks the visited cell and removes it from the current path, enabling other branches to reuse that cell — the classical backtracking mechanism. If no boundary-reaching path exists (i.e., the civilian is completely surrounded by fire), the algorithm reports failure and the player is notified.

2.3 Linked Lists

A singly linked list is a dynamic data structure in which each element (node) stores a value and a pointer to the next node in the sequence [1]. Unlike arrays, linked lists support $O(1)$ insertion and deletion at the head or at a known position without requiring element shifting, at the cost of $O(n)$ access by index and additional memory for pointers.

Linked lists are a natural fit for implementing queues, where elements are enqueued at the tail and dequeued from the head [5]. In this project, the C++ engine uses a linked list to model the unit dispatch queue. New firefighting units (ground crews, trucks, helicopters) are appended to the tail as they arrive on a turn-based schedule, and the player dequeues from the front when selecting a unit to deploy. This FIFO ordering preserves the arrival sequence of units, and the dynamic nature of the linked list accommodates the variable arrival rates defined by each difficulty level without pre-allocating a fixed buffer.

2.4 Binary Search Trees

A Binary Search Tree (BST) is a rooted binary tree in which every node satisfies the BST property: all keys in the left subtree are smaller than the node's key, and all keys in the right subtree are larger [1]. This ordering enables search, insertion, and deletion in $O(h)$ time, where h is the tree height. For a balanced BST, $h = O(\log n)$; in the worst case (a degenerate tree), $h = O(n)$.

In this project, each BST node stores the coordinates of a burning cell, its computed risk score, and its fire type. Crown fire cells receive a risk multiplier of $\times 1.5$ relative to normal fire cells, reflecting their more aggressive eight-directional propagation. The BST is updated after every turn as the fire spreads, new cells ignite, and extinguished cells are removed, keeping the risk ranking consistent with the current game state.

Self-balancing variants such as the AVL tree or Red-Black tree guarantee $O(\log n)$ height at all times through rotation operations [1]. For this project, a standard BST without automatic balancing was chosen. Given the maximum grid size of $8 \times 8 = 64$ cells, the number of nodes in the tree is at most 64, making the practical performance difference between a BST and an AVL tree negligible. The simpler BST implementation reduces code complexity and is sufficient for the scale of this application.

2.5 Summary

The literature establishes greedy algorithms and backtracking search as complementary paradigms: greedy methods are fast and irrevocable, suited to problems where local optimality is globally sufficient or acceptable; backtracking is exhaustive and reversible, suited to constraint-heavy search problems where correctness is required. The linked list and BST provide the dynamic data management needed to support these algorithms efficiently within the game engine.

Chapter 3

Objectives

3.1 General Objective

To design and develop a turn-based tactical simulation game on an 8×8 grid that demonstrates the practical integration of fundamental data structures and algorithms within a multi-layer software architecture. The system combines a C++ engine — managing a singly linked list for unit dispatch and a Binary Search Tree for fire risk ranking — with a Python algorithm module implementing a Greedy dispatcher for unit deployment and a Backtracking planner for civilian evacuation, and a Pygame graphical interface that renders the game state and animates player actions across three difficulty levels.

3.2 Specific Objectives

- Implement a **Greedy algorithm** that automatically dispatches firefighting units to the cell with the highest fire risk, based on data maintained in a Binary Search Tree.
- Implement a **Backtracking algorithm** that finds safe evacuation routes for civilians, avoiding all burning cells on the 8×8 grid.
- Design and implement a **singly linked list** to manage the unit dispatch queue with FIFO (First-In-First-Out) ordering.
- Design and implement a **Binary Search Tree (BST)** to maintain burning cells ordered by risk level, enabling O(log n) retrieval of the highest-risk cell.
- Establish a **modular architecture** separating the C++ game engine, Python algorithms, and Pygame interface, communicating via JSON.
- Develop **three difficulty levels** (Easy, Medium, Hard) with varying initial conditions for fires, civilians, units, and arrival rates.
- Incorporate **three gameplay variants** to add strategic depth: scarce resources (unit arrival timers), civilian priorities (different point values and movement speeds), and dynamic fire behavior (normal and canopy propagation).
- Build a **Pygame-based graphical interface** that renders the 8×8 grid, fires, civilians, unit queue, score HUD, and evacuation animations.
- Define a **JSON communication protocol** (input.json and state.json) to enable data exchange between the C++ engine and Python algorithms.

Chapter 4

Scope

4.1 Included in this Study

This report covers the full conceptual design of a turn-based tactical fire simulation game, encompassing the following elements:

- The design of the game mechanics, including turn structure, fire propagation rules, scoring system, and win/loss conditions across three difficulty levels (Easy, Medium, and Hard).
- The specification of the data structures used in the C++ engine: a singly linked list implementing the unit dispatch queue and a Binary Search Tree organising burning cells by risk score.
- The algorithmic design of the Greedy unit dispatcher and the Backtracking evacuation planner, including pseudocode, input/output definitions, and step-by-step logic.
- The proposed three-layer software architecture comprising the C++ engine, the Python algorithm module, and the Pygame graphical interface, and the JSON Bridge communication protocol connecting them.
- The three built-in game variants: scarce unit resources, civilian priority differentiation, and dynamic fire behaviour with two propagation modes.
- A critical discussion of design trade-offs, algorithmic limitations, and potential future improvements.

4.2 Excluded from this Study

The following aspects fall outside the scope of this report:

- **Full software implementation:** the game has not been coded or executed. No source code, compiled binaries, or runtime measurements are presented. All content is design and proposal only.
- **Empirical performance testing:** no benchmarks, execution time measurements, or memory profiling results are included, as the system has not been implemented.
- **User testing and playability evaluation:** no usability studies, player feedback sessions, or gameplay balancing tests have been conducted.
- **Network or multiplayer functionality:** the game is designed as a single-player local application. Online play, multiplayer modes, and networked communication are not considered.

- **Mobile or web deployment:** the proposed interface targets desktop environments via Pygame. Adaptation to mobile platforms, browsers, or game engines such as Unity or Godot is outside the scope of this work.
- **Advanced AI opponents:** the system does not include computer-controlled adversaries or AI players. All decisions are made by a human player, assisted by the Greedy and Backtracking algorithms acting as decision-support tools rather than autonomous agents.
- **Persistent storage and user accounts:** save files, leaderboards, player profiles, and session persistence are not part of the proposed design.

Chapter 5

Assumptions and Limitations

5.1 Assumptions

The following assumptions were made during the design of the game and the elaboration of this report:

- **Grid representation:** the game world is assumed to be fully representable as a discrete 8×8 grid, where each cell occupies exactly one unit of space. No partial occupancy, overlapping entities, or sub-cell movement is considered.
- **Fire propagation determinism:** fire is assumed to spread deterministically at the end of every turn, without probabilistic variation. A burning cell always ignites all valid neighbours according to its type (4 directions for normal fire, 8 directions for canopy fire), provided those neighbours are not already burning or protected by a Helicopter.
- **Instantaneous extinguishment:** it is assumed that deploying a unit to a burning cell extinguishes it entirely within the same turn, with no partial or multi-turn suppression process.
- **Static civilian positions:** civilians are assumed to remain in their starting cell until the player explicitly triggers an evacuation. They do not move autonomously, seek shelter, or react to nearby fire between player actions.
- **BST insertion order:** the Binary Search Tree is assumed to be rebuilt from the current fire state at the start of each turn. It is assumed that the insertion order produces a sufficiently balanced tree for the problem scale (n 64), making worst-case degeneration unlikely in practice.
- **JSON Bridge reliability:** communication between the C++ engine and the Python algorithm module via the JSON Bridge is assumed to be synchronous, lossless, and free of serialisation errors. No fault-tolerance or retry mechanism is considered in the design.
- **Single-threaded execution:** the entire game loop — engine update, algorithm execution, and UI rendering — is assumed to run on a single thread in a sequential, turn-based manner. Concurrency and race conditions are not addressed.
- **User competency:** it is assumed that the player understands the basic rules of the game before starting a session. No in-game tutorial, guided onboarding, or contextual help system is included in the proposed design.

5.2 Limitations

- **Design-only stage:** the most significant limitation of this work is that the game exists solely as a proposal. No implementation has been produced, which means that the correctness of the

algorithms, the performance of the data structures, and the playability of the game mechanics have not been empirically validated. All claims about algorithmic behaviour are based on theoretical analysis alone.

- **No gameplay balancing:** the scoring values, unit arrival intervals, civilian point weights, and loss thresholds defined in the design have not been tested through actual play sessions. It is possible that certain difficulty configurations are trivially easy or impossible to win, and adjustments would likely be needed after playtesting.
- **Greedy suboptimality:** the Greedy dispatcher does not guarantee a globally optimal sequence of deployments. In scenarios where long-term planning would yield a significantly better outcome than greedy local decisions — particularly when managing Helicopter units — the algorithm may lead the player towards avoidable defeat without the player being aware of the suboptimal choice.
- **Static pathfinding:** the Backtracking evacuator computes routes based on the fire state at the moment of the decision, without accounting for future fire propagation. For slow civilians such as the elderly type, this may result in routes that become unsafe before the civilian completes them, a limitation that cannot be resolved without a time-aware pathfinding approach.
- **Fixed grid size:** the design is tightly coupled to an 8×8 grid. Extending the system to larger grids would require reassessing the BST in favour of a self-balancing structure, and would expose the exponential worst-case complexity of the Backtracking algorithm, potentially requiring replacement with BFS or A*.
- **Time constraints:** the scope of this project was bounded by academic deadlines, which limited the depth of exploration into alternative algorithmic approaches, more sophisticated fire propagation models, and extended difficulty configurations. Some design decisions reflect pragmatic choices made under time pressure rather than exhaustive optimisation.

Chapter 6

Methodology

This chapter describes the proposed design of the tactical fire simulation game, covering the system architecture, the data structures selected for the C++ engine, the algorithmic logic governing unit deployment and civilian evacuation, the communication bridge between the C++ and Python layers, and the game rules that define each difficulty level. All content in this chapter represents the intended design and has not yet been fully implemented.

6.1 System Architecture

The system is divided into three independent layers that communicate through defined interfaces:

- **C++ Engine:** manages the game state, including the 8×8 grid, fire propagation logic, the unit dispatch queue (linked list), and the risk-ranking structure (BST). At the end of each turn the engine serialises the current game state to a JSON object and passes it to the Python layer.
- **Python Algorithm Module:** receives the JSON game state, executes the Greedy dispatcher or the Backtracking evacuator depending on the player's action, and returns the resulting decision (target cell or evacuation route) to the engine as a JSON response.
- **Pygame Interface:** reads the game state produced by the engine after each turn and renders the grid, fire cells, civilian positions, unit queue, score HUD, and evacuation animations. All player input is captured here and forwarded to the engine.

The JSON Bridge is the sole communication channel between C++ and Python. Each message contains the full grid state, the list of active fires with their types and risk scores, the positions and types of all civilians, and the contents of the unit dispatch queue. This stateless interface keeps the three layers decoupled and allows each to be developed and tested independently.

6.2 Data Structure Design

6.2.1 Unit Dispatch Queue — Singly Linked List

The unit dispatch queue is implemented as a singly linked list in C++. Each node in the list represents one available firefighting unit and stores the following fields:

- `unit_type` (string): one of `GRUPO`, `CAMION`, or `HELICOPTERO`.
- `arrival_turn` (int): the turn on which the unit became available.
- `*next` (pointer): reference to the next node in the queue, or `nullptr` if this is the last unit.

New units are appended at the tail as they arrive according to the turn-based schedule defined by the difficulty level. The player dequeues from the head when selecting a unit to deploy. This FIFO structure preserves arrival order and supports $O(1)$ enqueue and dequeue operations.

6.2.2 Risk-Ranking Structure — Binary Search Tree

The BST organises all currently burning cells ordered by their risk score. Each node stores:

- `risk_level` (float): the computed risk score, equal to the base propagation weight of the cell multiplied by 1.5 if the fire type is `copas` (crown fire).
- `cell_x`, `cell_y` (int): grid coordinates of the burning cell.
- `fire_type` (string): `normal` or `copas`.
- `*left`, `*right` (pointers): left and right child nodes.

The BST property (left subtree $<$ node $\leq$ right subtree by risk level) ensures that the maximum-risk cell is always the rightmost node, retrievable in $O(h)$ time. The tree is rebuilt after each turn to reflect newly ignited cells, extinguished cells, and updated risk scores caused by fire propagation.

6.3 Algorithm Design

6.3.1 Greedy Unit Dispatcher

When the player chooses to deploy a unit, the **DespachadorGreedy** algorithm is invoked. It takes the current BST and the selected unit type as input and returns the target cell coordinates.

1. Query the BST for the node with the maximum `risk_level` by traversing right-child pointers from the root.
2. If multiple cells share the maximum risk value, apply tie-breaking rules:
   (a) Prefer any cell whose `fire_type` is `copas`, as crown fires propagate more aggressively.
   (b) If no crown fire exists among the tied cells, select the cell with the smallest row index; if rows are equal, select the smallest column index.
3. Return the selected cell coordinates as the deployment destination.

The player retains strategic control over *which* unit to deploy and *when* to deploy it, but the destination is always determined by the Greedy algorithm. This design reflects the constraint that firefighting units in the field are dispatched to the most critical location rather than at the operator's discretion.

6.3.2 Backtracking Evacuation Planner

When the player chooses to evacuate a civilian, the **EvacuacionBacktracking** algorithm is invoked. It takes the civilian's starting cell, the obstacle matrix (cells currently on fire), and the civilian type as input, and returns the shortest safe path to any grid boundary cell, or reports failure if no such path exists.

The algorithm proceeds as follows:

1. Initialise a 8×8 boolean `visited` matrix to `false`, an empty `current_path` list, an empty `best_path` list, and `best_length` $\leftarrow \infty$.
2. Call the recursive `explore` function from the civilian's starting position.

3. Inside `explore(position)`:
   (a) Append `position` to `current_path`.
   (b) *Base case:* if `position` is on the grid boundary (row = 0, row = 7, column = 0, or column = 7), compare `len(current_path)` with `best_length`; if shorter, update `best_path` and `best_length`.
   (c) Mark `visited[position]` ← `true`.
   (d) For each of the four cardinal neighbours (up, down, left, right): if the neighbour is within the grid, is not on fire, and has not been visited, recurse into `explore(neighbour)`.
   (e) *Backtrack:* set `visited[position]` ← `false` and remove `position` from `current_path`.
4. After `explore` returns, if `best_path` is non-empty, return it with `success = true`; otherwise return an empty list with `success = false`.

The civilian type influences movement speed in the UI layer: an `ANCIANO` (elderly civilian) requires two turns to advance one cell along the returned path, while all other types advance one cell per turn. The algorithm itself is type-agnostic; speed is enforced by the engine when animating the route.

6.4 Game Rules and Difficulty Levels

6.4.1 Scoring System

Points are awarded and deducted according to the following rules:

- Extinguishing a normal fire cell: $+10 \times$ risk_level.
- Extinguishing a crown fire cell: $+15 \times$ risk_level.
- Evacuating a child: $+100$ points.
- Evacuating an adult: $+50$ points.
- Evacuating an elderly civilian: $+75$ points.
- Evacuating a family group: $+150$ points.
- Civilian burned: -100 points (civilian is removed from the map).
- All fires extinguished bonus: $+200$ points.
- All civilians evacuated bonus: $+300$ points.

6.4.2 Win and Loss Conditions

The player wins when all fires on the grid have been extinguished **and** all civilians have been evacuated. The player loses if either of the following conditions is met:

- More than half of the civilians present at the start of the level are burned. For example, with 4 initial civilians the third burn triggers defeat; with 3 civilians, the second burn triggers defeat.
- Fire spreads to cover more than 80% of the grid (64 cells), preventing indefinitely prolonged games.

Table 6.1: Difficulty level configurations

Parameter	Easy	Medium	Hard
Normal fires	2	2	2
Canopy fires	0	1	3
Civilians	2	3	4
Initial units	Group, Truck, Helicopter	Group, Truck, Helicopter	Group, Truck
Group arrival (turns)	2	3	4
Truck arrival (turns)	4	5	6
Helicopter arrival (turns)	6	7	8
Fire propagation	Normal only	Mixed	Aggressive

6.4.3 Difficulty Levels

Table 6.1 summarises the configuration of each difficulty level.
On the Easy level, only normal fire (4-directional propagation) is present. On Medium, one crown fire cell is introduced, adding 8-directional propagation and a $\times 1.5$ risk weight. On Hard, three crown fires are active from the start, no helicopter is provided initially, and all unit arrival intervals are at their longest, placing maximum pressure on the player's resource management.

6.5 Turn Structure

Each game turn follows this sequence:

1. **Fire propagation:** the engine expands each active fire cell to its neighbours according to its type (4 or 8 directions). Newly ignited cells are inserted into the BST with their computed risk scores.

2. **Unit arrival:** any units scheduled to arrive on the current turn are appended to the tail of the linked list queue.

3. **Player actions:** the player may perform one or more actions while units remain in the queue. Each action is either:

   - *Deploy a unit* — select a unit type from the queue; the Greedy algorithm determines the target cell; the unit is dequeued and the fire at the target cell is extinguished.
   - *Evacuate a civilian* — select a civilian; the Backtracking algorithm computes the shortest safe route; the civilian begins moving along the route at their type-defined speed.

4. **State update:** the engine updates the BST, checks win/loss conditions, serialises the new game state to JSON, and forwards it to the Pygame interface for rendering.

Chapter 7

Implementation & Results

Chapter 8

Conclusions

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[2] J. Kleinberg and E. Tardos, *Algorithm Design*. Boston, MA: Addison-Wesley, 2005.

[3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

[4] S. S. Skiena, *The Algorithm Design Manual*, 2nd ed. London: Springer, 2008.

[5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2011.

[6] B. Stroustrup, *The C++ Programming Language*, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2013.

[7] M. Lutz, *Learning Python*, 5th ed. Sebastopol, CA: O'Reilly Media, 2013.

[8] Pygame Community, "Pygame — Python game development," https://www.pygame.org, 2024, accessed: 2026.

[9] Ecma International, "ECMA-404: The JSON data interchange standard," https://www.ecma-international.org/publications-and-standards/standards/ecma-404, 2017, accessed: 2026.